Project GEWN

USER_FLOW.md

> Comprehensive user flow specification for the GEWN AI-Native Desktop Copilot Platform. Intended audience: developers, designers, and product contributors.

Table of Contents

---

1. How Users Interact with GEWN

GEWN lives on top of the user's desktop as a floating, always-on-top overlay. Users can interact with it through three primary entry points: **voice**, **chat**, and **keyboard shortcuts**. All three entry points funnel into the same backend AI engine.

```
User
  │
  ├──── 🎙️ Voice        →  Wake word or hold-to-talk
  ├──── 💬 Chat          →  Type into the floating chat panel
  └──── ⌨️ Shortcut      →  Keyboard shortcut opens/focuses GEWN
```

Interaction Modes

Mode	Trigger	Best For
Voice	"Hey GEWN" / hold key	Hands-free, quick commands
Chat	Click overlay or shortcut	Detailed requests, code help
Overlay Tap	Click on screen element	Visual assistance, error help
Background Agent	Scheduled / event-driven	Autonomous recurring tasks

General Interaction Principles

- GEWN is always available — it does not need to be launched per session.
- It maintains context across the current session and recalls long-term memory across sessions.
- It asks for confirmation before executing any destructive or sensitive action.
- All interactions are streamed — responses appear progressively, not all at once.
- GEWN can be minimized to a small icon and recalled instantly.

---

2. Voice Interaction Flow

2.1 Standard Voice Flow

```
[User]
  |
  ▼
Wake Word Detected: "Hey GEWN"
  |
  ▼
GEWN Activates – Visual & Audio Cue (pulse animation + chime)
  |
  ▼
Speech-to-Text (Whisper / Deepgram)
  |
  ▼
Transcribed Intent Sent to AI Engine
  |
  ▼
AI Processes Intent
  |
  ├──── Simple Response → Text-to-Speech (ElevenLabs / Coqui TTS)
```

```
   ├──── Task Detected   → Confirm → Execute → Speak Result
   └──── Clarification Needed → GEWN Asks Follow-up Question
   |
   ▼
Voice Output + Optional Overlay Update
```

2.2 Example Interaction

> **User:** "Hey GEWN, summarize the last document I was working on."
>
> **GEWN:** *(activates, listens)* "Sure — you were working on `research_notes.pdf`. It covers three main themes: climate policy, renewable energy funding, and carbon offset markets. Want me to go deeper on any of these?"

2.3 Real-Time Voice Mode

For extended conversations, GEWN enters **Real-Time Voice Mode**, a continuous dialogue state:

```
User activates Real-Time Mode (button or shortcut)
   |
   ▼
GEWN Opens Persistent Listening Channel (WebSocket audio stream)
   |
   ▼
User speaks naturally — GEWN detects end of speech automatically
   |
   ▼
GEWN responds while still listening (interruption-aware)
   |
   ▼
User says "Stop listening" or presses shortcut to exit
```

2.4 Edge Cases

Scenario	GEWN Behavior
No speech detected for 5 seconds	Plays soft chime, closes listening window
Ambient noise triggers wake word	Audio gate re-confirms intent before acting
User speaks over GEWN response	GEWN pauses, listens to the new input
STT confidence below threshold	GEWN says "I didn't catch that — could you repeat?"

Scenario	GEWN Behavior
Mic not available / permission denied	Visual fallback: prompts user to type instead

2.5 Failure Handling

- If STT service is unavailable → fallback to chat input with a notification.
- If TTS service fails → response is shown as text only, with a small audio error icon.
- If wake word model crashes → GEWN restarts the detection subprocess silently.

---

3. Chat Interaction Flow

3.1 Standard Chat Flow

```
[User opens GEWN overlay or presses shortcut]
   |
  ▼
Chat Panel Appears (floating, glassmorphism UI)
   |
  ▼
User Types Message
   |
  ▼
Message Sent to FastAPI Backend (via WebSocket)
   |
  ▼
Memory Context Retrieved (short-term + relevant long-term)
   |
  ▼
AI Engine Processes Request (LLM + agent routing)
   |
  ▼
Response Streams Back to Chat Panel (token by token)
   |
  ▼
Response Rendered (Markdown, code blocks, inline results)
   |
  ▼
Action Buttons Shown if Applicable (e.g. "Run this", "Save note")
```

3.2 Example Interactions

Coding help:

> **User:** "Explain why this function returns `None` even though I have a return statement."
>
> **GEWN:** *(detects screen context via OCR)* "Your function has an early `return` inside the `if` block, but if the condition is false, execution falls through without hitting it. Here's the fix: …"

Research query:

> **User:** "What are the main causes of context window limitations in large language models?"
>
> **GEWN:** *(searches memory / performs web lookup if needed)* "Context window limits stem from three core factors: quadratic attention complexity, positional encoding decay, and memory allocation costs…"

3.3 Chat Session States

```
Idle → Typing → Waiting for Response → Streaming Response → Done
  |                                                          |
  └──────────────────────────── User can type new message at any time ──────────────┘
```

3.4 Edge Cases

Scenario	GEWN Behavior
User sends empty message	Ignored — input field shakes gently
Response takes too long (> 8s)	Shows animated "thinking" indicator
LLM returns error	Displays: "Something went wrong. Try again?" with retry button
User asks GEWN to do something ambiguous	Asks a single clarifying question before proceeding
User sends a very long message (> 4000 chars)	Accepted; GEWN chunks it internally if needed

3.5 Failure Handling

- API timeout → retry once automatically; show error toast on second failure.
- No internet connection → notify user, offer local-only LLM fallback if Privacy Mode is enabled.
- LLM hallucination risk detected → GEWN prefixes uncertain claims with "I'm not fully sure, but…"

---

4. Command Execution Flow

Commands are natural-language instructions that trigger specific system-level or AI-level actions.

4.1 Command Execution Flow

```
[User types or speaks a command]
   |
  ▼
AI Engine Classifies Intent
   |
   ├──── Simple Info Request     → Answer directly (no execution)
   ├──── Automation Command      → Route to Automation Engine
   ├──── File Operation          → Route to File System Tools
   ├──── Browser Task            → Route to Playwright / Browser Agent
   └──── Compound Task           → Route to Planning Engine (multi-step)
   |
  ▼
Risk Level Assessed
   |
   ├──── Low Risk (read-only, reversible)  → Execute immediately
   └──── High Risk (delete, system change) → Permission Approval Flow
   |
  ▼
Tool Executes Action
   |
  ▼
Result Streamed Back to User
   |
  ▼
Confirmation Message + Optional Undo / Log Entry
```

4.2 Example Commands

Command	Type	Execution Path
"Open VS Code"	App control	Electron API / PyAutoGUI
"Search YouTube for lo-fi playlists"	Browser	Playwright agent
"What's the weather in Mumbai?"	Info query	Web search / API
"Delete all .tmp files in Downloads"	File operation	Permission required first
"Set up my coding workspace"	Compound workflow	Planning engine

4.3 Command Classification Logic

```
Input Text
   │
   ▼
Intent Parser (LLM + classifier)
   │
   ├──── Verb detected (open, delete, search, create) → Action intent
   ├──── Question form (what, how, why) → Info intent
   ├──── Multi-part sentence → Compound intent → decompose into sub-tasks
   └──── Ambiguous → Ask clarifying question
```

4.4 Edge Cases

Scenario	GEWN Behavior
Command cannot be executed (app not installed)	Reports specific error, suggests alternative
Command conflicts with OS permissions	Explains the restriction, offers workaround
User cancels mid-execution	Stops cleanly, rolls back partial changes if possible
Command loop detected (recursive automation)	Breaks loop, alerts user

---

5. File Management Flow

5.1 Standard File Operation Flow

```
[User requests a file action]
   |
   ▼
AI Parses Request
   |
   ▼
Target Location Resolved (path, folder, recent files, or semantic search)
   |
   ▼
Action Type Determined:
   ├─── Create        → Generates file at path
   ├─── Rename        → Applies new name
   ├─── Move / Copy   → Confirms destination
   ├─── Organize      → Groups by type / date / content
   └─── Delete        → Triggers Permission Approval Flow
   |
   ▼
Confirmation Preview Shown to User (affected files listed)
   |
   ▼
User Approves → Action Executed
   |
   ▼
Result Reported + Undo Option Offered (within session)
```

5.2 Example Interactions

Organize downloads:

> **User:** "Organize my Downloads folder."
>
> **GEWN:** "I found 84 files in Downloads. Here's my proposed organization:
>
> - `/Images/` → 23 files
> - `/Documents/` → 41 files
> - `/Archives/` → 12 files
> - `/Installers/` → 8 files
>
> Proceed?"

Semantic file search:

User: "Find my notes from last month's project brainstorm."

GEWN: *(searches vector memory + file system)* "I found `brainstorm_nov14.md` and `project_ideas_v2.docx` in `/Documents/Projects/`. Open one?"

5.3 Smart Organization Logic

```
File System Scan
   |
   ▼
File Metadata Read (name, extension, modified date, size)
   |
   ▼
Content Embeddings Generated (for documents, text files)
   |
   ▼
Semantic Clustering → Proposed Folder Structure
   |
   ▼
Preview Shown → User Approves → Files Moved
```

5.4 Edge Cases

Scenario	GEWN Behavior
File already exists at destination	Asks: rename, skip, or overwrite
File is in use by another application	Reports which app holds the file
User targets system-protected directory	Refuses, explains restriction
Target path doesn't exist	Offers to create the directory first
Operation fails midway	Reports partial completion, lists remaining files

5.5 Failure Handling

- Permission errors → explain and offer to retry with elevated rights (OS prompt).
- Disk full → warn before operation starts, suggest cleanup options.
- File locked → report and retry after brief delay (up to 3 times).

---

6. Automation Workflow Flow

6.1 Standard Workflow Flow

```
[User requests multi-step workflow]
   |
  ▼
AI Engine Detects Compound Intent
   |
  ▼
Planning Engine Decomposes into Steps
   |
  ▼
Step Sequence Displayed to User for Confirmation
   |
  ▼
User Approves Plan
   |
  ▼
Steps Execute Sequentially (or in parallel where safe)
   |
  ▼
Progress Shown in Real-Time (step counter in overlay)
   |
  ▼
Each Step Result Logged
   |
  ▼
Final Summary Delivered to User
```

6.2 Example Workflow

> **User:** "Prepare my coding workspace."

GEWN's Plan:

```
Step 1 – Open VS Code                        ✓
Step 2 – Open Terminal in project folder     ✓
Step 3 – Open Chrome with localhost:3000     ✓
Step 4 – Open Notion notes for this project  ✓
Step 5 – Open Spotify with focus playlist    ✓
```

> **GEWN:** "Your workspace is ready. All 5 steps completed in 4 seconds."

6.3 Workflow Memory & Presets

When a user runs the same workflow multiple times, GEWN learns it:

```
Workflow Executed
   |
   ▼
Workflow Saved to Long-Term Memory
   |
   ▼
Next Time User Asks → GEWN Suggests: "Use your saved 'Coding Workspace' preset?"
   |
   ▼
User Confirms → Workflow Runs Instantly
```

6.4 Scheduled Workflows

```
User: "Every Monday morning at 9am, open my emails, calendar, and task list."
   |
   ▼
GEWN Creates Scheduled Trigger (cron-style)
   |
   ▼
At Trigger Time → Workflow Executes Automatically
   |
   ▼
User Notified of Completion (or any errors)
```

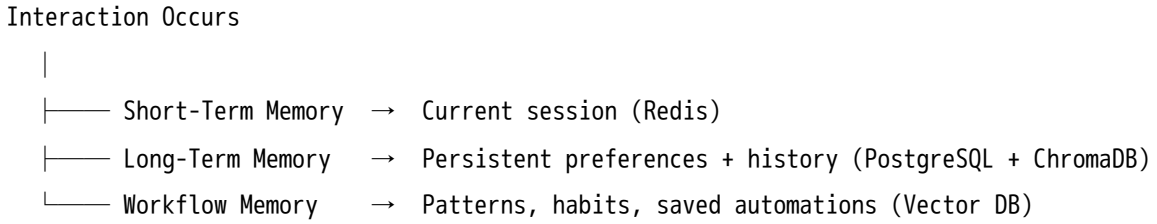
6.5 Edge Cases

Scenario	GEWN Behavior
One step fails mid-workflow	Pauses, reports failure, asks to skip or abort
Required app is not installed	Reports and skips that step, completes the rest
Workflow takes too long	Shows live progress; offers to run in background
User cancels during execution	Stops cleanly, logs which steps completed
Conflict between workflow steps	Re-orders or flags for user resolution

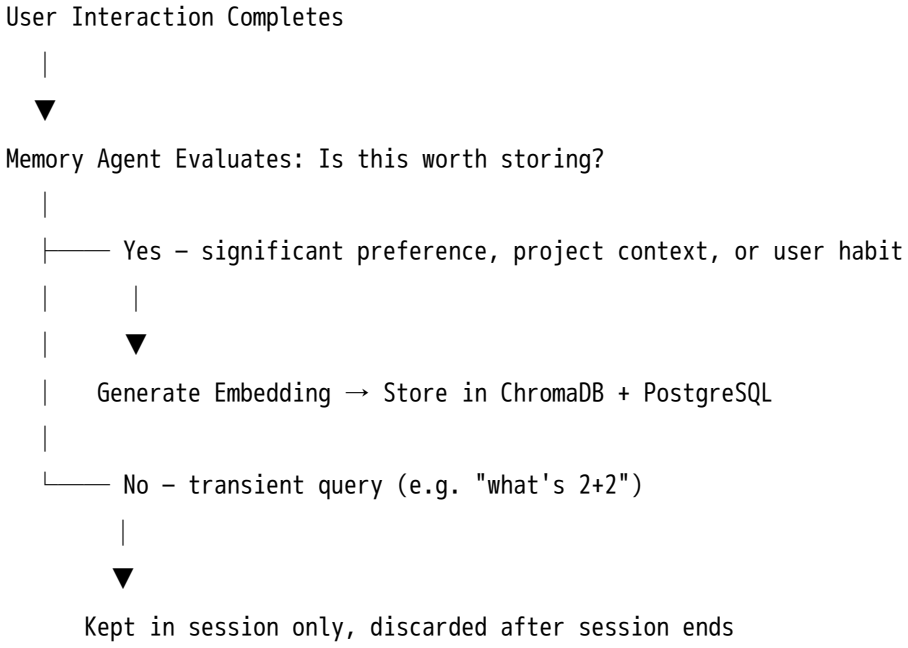
---

7. AI Memory Flow

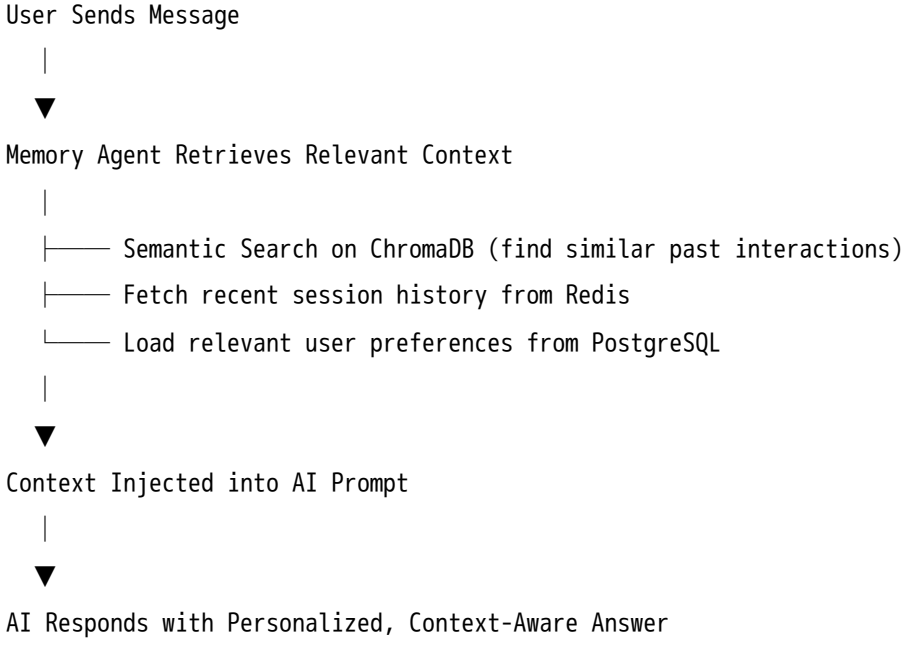
7.1 Memory Layers Overview



7.2 How Memory is Written



7.3 How Memory is Retrieved



7.4 Example Memory Scenarios

Preference recalled:

> User once said: "I prefer TypeScript over JavaScript."
>
> Later: **User:** "Write me a utility function for debouncing."
>
> **GEWN:** *(recalls preference)* "Here's a debounce function in TypeScript: …"

Project context recalled:

> **User:** "Continue where we left off with the API design."
>
> **GEWN:** "Sure — last session we were designing the `/users` endpoint. We'd finished authentication flow and were about to handle pagination. Want to pick up there?"

7.5 Second Brain (Knowledge Store)

Users can explicitly save knowledge to GEWN's Second Brain:

```
User: "Remember this – my preferred DB for side projects is SQLite."
  |
  ▼
GEWN: "Got it. Saved to your preferences."
  |
  ▼
Stored with tag: [preference / databases]
  |
  ▼
Retrieved automatically in future DB-related conversations
```

7.6 Edge Cases

Scenario	GEWN Behavior
Memory conflict (old vs new preference)	Uses most recent; surfaces both if ambiguous
Memory retrieval returns irrelevant context	Memory is not injected; response based on current input
User asks GEWN to forget something	Entry removed from vector store + relational DB

Scenario	GEWN Behavior
Memory store unreachable	Falls back to session-only mode with a notice
Privacy mode enabled	No cloud memory writes; local storage only

---

8. Vision / Screen Understanding Flow

8.1 Standard Screen Awareness Flow

```
[Trigger: User request, hotkey, or GEWN detects a relevant event]
  │
  ▼
Screenshot Captured (current screen or active window)
  │
  ▼
OCR Extraction (Tesseract / PaddleOCR)
  │
  ▼
Extracted Text + Image Sent to Multimodal LLM
  │
  ▼
AI Understands Visual Context
  │
  ├──── Coding error visible  → Explain and suggest fix
  ├──── Form visible          → Offer to fill it
  ├──── Document visible      → Summarize or answer questions
  └──── UI confusion apparent → Provide interactive overlay guidance
  │
  ▼
Response Delivered (text in chat + optional visual overlay)
```

8.2 Example Interactions

Error detection:

> GEWN detects a red stack trace on screen.
>
> **GEWN (proactive):** "I see a `TypeError` in `utils.py` on line 42 — looks like you're passing a string where an integer is expected. Want me to show you the fix?"

UI guidance:

> **User:** "What does this button do?" *(clicks on screen element)*
>
> **GEWN:** *(captures region, analyzes UI)* "That's the 'Publish' button for your blog post. It will make the post publicly visible. Would you like to preview first?"

8.3 Coding Error Detection Flow

```
Active Window = Code Editor
   │
  ▼
Periodic Screen Capture (low-frequency, opt-in)
   │
  ▼
OCR Extracts Visible Code + Error Messages
   │
  ▼
Error Pattern Matching (stack trace, linter warning, compile error)
   │
  ▼
AI Analyzes Error in Code Context
   │
  ▼
GEWN Surfaces: Explanation + Suggested Fix + "Apply Fix" button
```

8.4 Visual Overlay Guidance Flow

```
User asks for help with something visible on screen
   │
  ▼
GEWN Captures Relevant Screen Region
   │
  ▼
AI Identifies UI Elements (buttons, fields, panels)
   │
  ▼
Overlay Rendered on Top of Screen:
   ├──── Arrows pointing to relevant elements
   ├──── Labels explaining what each part does
   └──── Step numbers guiding the user through the action
```

```
    |
    ▼
User Completes Action → Overlay Dismissed
```

8.5 Edge Cases

Scenario	GEWN Behavior
Screen content is purely graphical (e.g. game)	Reports limited OCR ability; uses visual AI for best-effort analysis
Multiple monitors active	Captures active window monitor by default; user can specify
Screen locked or private	Skips capture, waits for screen to be accessible
OCR confidence is low	Reports uncertainty: "I can make out most of this but some text is unclear."
Sensitive content on screen (passwords, banking)	GEWN does not store or log this content; analysis is ephemeral

8.6 Privacy Rules for Vision

- Screenshots are never stored permanently unless the user explicitly saves them.
- OCR output is processed in memory only.
- No screen content is transmitted to external servers unless the user enables cloud vision.
- In Privacy Mode: all vision processing runs on-device.

---

9. Permission Approval Flow

GEWN requires explicit user approval for any action that is irreversible, sensitive, or system-level.

9.1 Permission Levels

Level	Example Actions	Default Behavior
Safe	Read files, open apps, search web	Execute without prompt
Moderate	Move files, create folders, paste clipboard	Brief confirmation toast
High	Delete files, run terminal commands	Full approval dialog

Level	Example Actions	Default Behavior
Critical	Modify system settings, access credentials	Requires typed confirmation or biometric

9.2 Approval Flow

```
Action Requested
   |
   ▼
Risk Classifier Assigns Level
   |
   ▼
[Level: Safe]
   └──── Execute immediately

[Level: Moderate]
   └──── Show non-blocking toast: "Moving 3 files to /Archives. Undo?" → Auto-proceeds in 5s

[Level: High]
   └──── Show blocking dialog:
         "⚠️ GEWN wants to delete 12 files permanently.
          Affected files: [list shown]
          [Confirm]  [Cancel]"

[Level: Critical]
   └──── Show full confirmation modal:
         "🔐 This action requires your approval.
          Action: Modify system startup items.
          Type 'confirm' to proceed or press Cancel."
```

9.3 Undo System

For approved moderate actions, GEWN maintains an undo buffer:

```
Action Executed
   |
   ▼
Undo Record Created (action log + reverse operation)
   |
   ▼
Undo Option Shown for 30 Seconds
```

```
   │
   ├─── User clicks Undo → Reverse operation executed
   └─── Time expires → Undo record archived (recoverable within session)
```

9.4 Example Approval Dialogs

File deletion:

```
┌──────────────────────────────────────────┐
│  ⚠️ Confirm Action                       │
│                                          │
│  GEWN wants to delete the following:     │
│                                          │
│    • old_backup.zip   (234 MB)           │
│    • temp_notes.txt   (4 KB)             │
│    • draft_v1.docx    (18 KB)            │
│                                          │
│  This action cannot be undone.           │
│                                          │
│          [Cancel]    [Delete Files]      │
└──────────────────────────────────────────┘
```

Terminal command:

```
┌──────────────────────────────────────────┐
│  🖥️ Terminal Command                     │
│                                          │
│  GEWN wants to run:                      │
│                                          │
│    $ npm run build && npm run deploy     │
│                                          │
│          [Cancel]    [Run Command]       │
└──────────────────────────────────────────┘
```

9.5 Edge Cases

Scenario	GEWN Behavior
User ignores approval dialog	Action does not proceed; times out after 60s
User approves but action fails	Reports failure, logs the attempt

Scenario	GEWN Behavior
Same action approved repeatedly	GEWN can offer "Always allow for this workflow"
User denies approval	Action is cancelled; GEWN asks if they want an alternative approach
Approval UI crashes	Action is blocked by default (fail-safe)

Summary

Project GEWN is designed around the principle that AI interaction should feel effortless, transparent, and trustworthy. Every flow balances:

- **Speed** — low-latency streaming responses and instant execution for safe tasks.
- **Safety** — tiered permission approvals and undo support for any consequential action.
- **Context** — persistent memory and screen awareness making every interaction smarter than the last.
- **Clarity** — users always know what GEWN is doing, why, and how to stop it.

These flows form the foundation of the GEWN user experience and should guide both frontend design decisions and backend architecture choices as the system evolves.